Moving Through Unknown Maps: D* Replanning and Bug Algorithms

Contents

1. Overview . . . 2
2. Planning on a map that may be wrong . . . 3
   - 2.1 Planning on the map until a sensor contradicts it . . . 4
   - 2.2 Why plan from the door . . . 4
3. Repairing the numbers: D* . . . 4
   - 3.1 Which numbers the box makes wrong . . . 4
   - 3.2 Fixing them in order . . . 5
   - 3.3 The standard terms . . . 8
4. When repairing costs less than a new search . . . 8
5. No time to plan to the door: real-time search . . . 11
   - 5.1 Learning the guesses: LRTA* . . . 11
   - 5.2 Looking further: RTAA* . . . 14
6. No map at all: bug algorithms . . . 15
7. Summary . . . 20
8. Sources . . . 21
9. Key terms . . . 22

Where this fits:

- **Builds on:** A* search; Admissible heuristic; Growing obstacles by the robot's radius; Value iteration on a grid.
- **Compare with:** A* search.

1. Overview

Key point: A plan is only as good as the map, and a box the map does not show makes the robot's numbers wrong. Instead of searching again from scratch at every surprise, D* repairs only the numbers the box changes; with no time for a full search, real-time search moves after searching a few cells; with no map at all, bug algorithms follow the obstacle's edge.

So far the robot can reach the door from any cell of its map: every free cell holds its cost-to-go (G-2701), its shortest distance to the door's cell G, and the robot walks downhill on those numbers, 3.414 m from the start cell S.

The remaining problem: the numbers are only as correct as the map. Someone has left a box, 1.0 m by 0.5 m, on the floor at x 2.0 to 3.0 m and y 2.0 to 2.5 m, across the robot's route, and the map shows free floor there.

The robot's sensor sees only the 8 cells around it, 0.5 m. Figure 1 plays what happens: after one move the robot sees the box, and its cell's number, 2.707 m, is wrong, because the way round the box is 4.414 m.

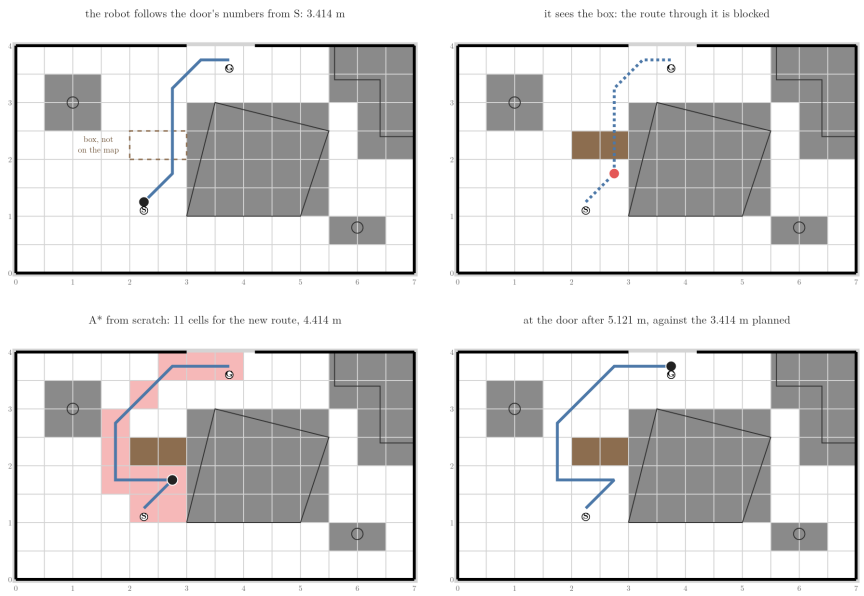


Figure 1: The room's 0.5 m grid (grey: blocked cells, with the true furniture outlined), the start cell S and the door's cell G. The robot (black dot, drawn at the centre of its cell) follows the door's numbers along its 3.414 m route; the box (dashed outline) stands on the route, but the map shows free floor there. After one move the box is 0.5 m away and the robot sees its two cells (brown): the route through them is blocked, and its cell's number, 2.707 m, is wrong. A* from scratch searches 11 cells (light red) for a new route of 4.414 m; the robot reaches the door after 5.121 m, against the 3.414 m planned

The simplest fix is to run A* search (G-2577) again from the robot's cell, here 11 cells, and to do that at every surprise. On a map of 0.1 m cells the robot sees the box piece by piece, plans again 7 times, and A* from scratch takes 596 cells off its open lists; a method that keeps and repairs its numbers takes 387 (notebook). Every number below is computed in the notebook `RO-049-replanning-d-star-and-bug-algorithms.ipynb`. This Note builds:

- planning on a map that may be wrong: trust it, move, look, plan again, and why to plan from the door (Section 2);
- D*, which repairs only the numbers a change makes wrong, and D* Lite (Section 3);
- when repairing costs less than searching again, measured on finer maps (Section 4);
- real-time search, LRTA* and RTAA*, for when there is no time to plan all the way to the door (Section 5);
- bug algorithms, for a robot with no map at all (Section 6).

2. Planning on a map that may be wrong

> **Key point:** The robot treats every cell it has not seen blocked as free, plans, moves one cell, looks, and plans again when what it sees differs from its map. Planning from the door keeps the numbers right while the robot moves; only a change in the map makes some of them wrong.

2.1 Planning on the map until a sensor contradicts it

The box in the opening is a cell the map gets wrong. The robot cannot wait for a perfect map, so it plans with the one it has: it treats every cell its map shows free, or has never seen, as free until a sensor shows otherwise (CMU incremental, slide 13; Koenig and Likhachev 2002).

Treating unseen cells as free is optimistic: the first plan is the shortest route that could exist, and every surprise can only make the route longer.

The robot then runs a loop (CMU incremental, slides 2 and 3):

1. plan a route to the door with the map as it stands;
2. move one cell along it;
3. look at the 8 cells around it and mark any blocked cell on the map;
4. if a marked cell changes the plan, plan again; go back to step 2.

Computing a new plan while moving, because the map, the robot's position or the world has changed, is **replanning** (G-2711) (CMU incremental, slide 2). In Figure 1 the robot replanned once, at the cell x 2.5 to 3.0 m, y 1.5 to 2.0 m, and A* from scratch took 11 cells. A* keeps nothing from one search to the next: each search starts from scratch (Koenig and Likhachev 2002).

2.2 Why plan from the door

A search from the robot measures every number from the robot's cell, so the numbers are out of date as soon as it moves. A search from the door measures every number to the door, and the door does not move: as the robot drives, those numbers stay right (CMU incremental, slide 8). The cost-to-go map the robot walks down is such a backward search (G-2583), run from the door.

The robot's numbers therefore become wrong only when the map changes, and then only the numbers whose shortest route ran through the change. Which numbers are those, and how can the robot fix only them?

3. Repairing the numbers: D*

> **Key point:** Each cell's number should equal its lookahead, the smallest move plus neighbour's number, and the box breaks that at only 5 cells. D* queues those cells and fixes them in order, raising the numbers that relied on the box and lowering them to the way round, until the robot's cell is right: 12 steps take it from 2.707 to 4.414, and 68 numbers are never touched.

3.1 Which numbers the box makes wrong

The idea. In value iteration (G-2704) each cell's number came to equal the best offer from its neighbours: the smallest move plus neighbour's number. The box breaks that equality

only where an offer came through the box. To find the wrong numbers, each cell next to the box compares its number with its best offer now. With:

- s is a cell, and n one of its neighbours;
- $g(s)$ is the number the cell holds, its cost-to-go to the door as last computed, in m;
- $c(s, n)$ is the length of the move from s to n: 0.5 m straight, 0.707 m diagonal, infinity if a box cell or the corner rule now forbids it;
- $\text{rhs}(s)$ is the cell's lookahead, its best offer from its neighbours, in m (0 for G).

$$\text{rhs}(s) = \min_{n} \Big(c(s, n) + g(n) \Big)$$

The name rhs, "right-hand side", is D* Lite's (Koenig and Likhachev 2002). A cell whose number differs from its lookahead disagrees with what its neighbours now offer, so its number needs fixing (Koenig and Likhachev 2002).

On our numbers. The robot's cell held 2.707: 0.5 m up into the box's right cell, which held 2.207. With the box blocked, its offers are (notebook):

Move from the robot's cell	Move (m)	Neighbour's number (m)	Offer (m)
left	0.5	2.914	3.414
down	0.5	3.207	3.707
diagonally down and left, to S	0.707	3.414	4.121

The lookahead is the smallest, 3.414, not 2.707, so the robot's cell disagrees with its lookahead. Four more cells do: the two box cells (lookahead infinity), the cell left of the robot (2.914 against a lookahead of 3.207) and the cell left of the box (2.621 against 2.914).

But even 3.414 is wrong: the offer from the cell on the robot's left, 2.914, also ran through the box. The numbers must be fixed in the right order.

3.2 Fixing them in order

The mechanism. The cells that disagree with their lookahead wait in a priority queue (G-2568). The cell with the smallest key comes off first, and is fixed in one of two ways (Koenig and Likhachev 2002):

- **if its lookahead is smaller than its number**, a shorter way has opened: the number falls to the lookahead and is final, like a vertex settled by Dijkstra's algorithm (G-2565);
- **if its lookahead is larger**, its way has become longer: any old offer may itself have run through the change, so its number rises to infinity, and it goes back into the queue to fall to its new lookahead later.

After either change, every neighbour's lookahead is recomputed, and any neighbour that disagrees joins the queue. The key orders the queue like A*'s open list, so the cells on the robot's side are fixed first. With:

- $h(s)$ is the octile distance (G-2571) between the robot's cell and s, the guess of the distance between them, in m.

$$\text{key}(s) = \min\Big(g(s), \text{rhs}(s)\Big) + h(s)$$

Ties go to the smaller $\min(g, \text{rhs})$. The repair stops when the robot's cell agrees with its lookahead and no queued cell has a smaller key than the robot's cell (Koenig and Likhachev 2002). D* Lite also adds a correction to the keys each time the robot moves, so that keys made before and after a move stay comparable; in Figure 2 the queue is empty when the robot sees the box, so the correction does not change the order.

Figure 2 runs the repair one cell per frame, starting from the full number map, every cell's cost-to-go before the box was seen. Watch the first six frames raise numbers to infinity, spreading down and left from the box, and then frame "7" start the falls: the cell left of the box finds a way round, 0.5 m up to a cell whose 2.414 never changed.

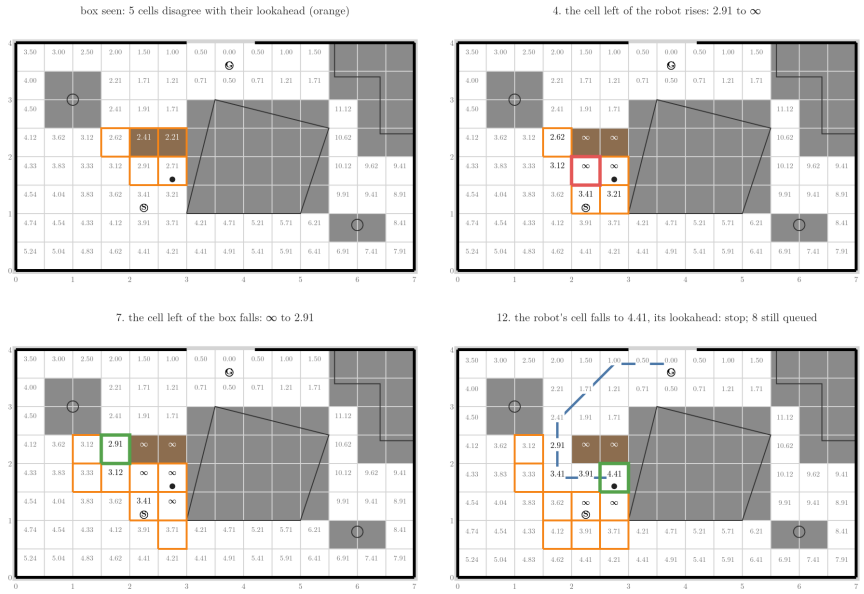


Figure 2: D* Lite repairing the door's numbers after the robot, at x 2.5 to 3.0 m, y 1.5 to 2.0 m (black dot), sees the box's two cells (brown); each cell shows its number, rounded to 0.01 m (grey: untouched). Orange outline: a cell whose number disagrees with its lookahead, waiting in the queue. Each frame takes one cell off the queue: a rise to infinity (red outline) or a fall to the cell's lookahead (green outline). Steps 1 to 6 raise the box's two cells, the robot's cell, the cells left of and below the robot, and the cell left of the box; step 7 lowers the cell left of the box to 2.91; steps 8 to 12 raise and lower the cells on the way round. The last frame: the robot's cell holds 4.41, equal to its lookahead, so the repair stops with 8 cells still queued, and the robot's new route (blue) goes left round the box

The 12 steps, in order (notebook):

1. the box's right cell rises from 2.207 to infinity;
2. the robot's cell rises from 2.707 to infinity;
3. the box's left cell rises from 2.414 to infinity;
4. the cell left of the robot rises from 2.914 to infinity;
5. the cell below the robot rises from 3.207, because its way ran through the robot's cell;
6. the cell left of the box rises from 2.621, because its diagonal past the box's corner is no longer allowed;
7. the cell left of the box falls to 2.914;
8. the cell below it, x 1.5 to 2.0 m, y 1.5 to 2.0 m, rises from 3.121;
9. S rises from 3.414;
10. the cell x 1.5 to 2.0 m, y 1.5 to 2.0 m falls to 3.414;
11. the cell left of the robot falls to 3.914;
12. the robot's cell falls to 4.414, its lookahead: the repair stops.

Why 4.414 is right. Each fall adds one move to a number that is already right, back to a number the box never touched: the cell above the cell left of the box kept its 2.414. Our numbers first, then the same in symbols, for a chain of cells s_1 (the robot's cell), s_2, s_3, s_4, s_5, each taking its number from the next:

$$g(s_1) = 0.5 + 3.914 = 4.414$$

$$g(s_1) = c(s_1, s_2) + g(s_2)$$

$$g(s_2) = 0.5 + 3.414 = 3.914$$

$$g(s_2) = c(s_2, s_3) + g(s_3)$$

$$g(s_3) = 0.5 + 2.914 = 3.414$$

$$g(s_3) = c(s_3, s_4) + g(s_4)$$

$$g(s_4) = 0.5 + 2.414 = 2.914$$

$$g(s_4) = c(s_4, s_5) + g(s_5)$$

The robot's new number is the length of a real route round the box, and it equals what A* from scratch found, 4.414 m (notebook). The number is the shortest because the falls

come off the queue in key order, the order that makes each settled number final in Dijkstra's algorithm (Koenig and Likhachev 2002).

Only 8 cells' numbers changed, and 68 of the 76 were never touched (notebook). Eight cells are still queued when the repair stops, S among them: their numbers are wrong, but the robot does not need them unless it goes there.

3.3 The standard terms

Keeping the cost-to-go from a backward search and, when the sensed costs differ from the map, putting the affected cells back in the priority queue until the robot's own cost-to-go is known again is **D*** (G-2713), "dynamic A*", published by Anthony Stentz in 1994 (LaValle §12.3.2; Stentz 1994; Wikipedia, D*). LaValle calls it "a dynamic version of the backward variant of Dijkstra's algorithm", and states that at any time the robot's motions are optimal for the information it has.

The repair we ran is **D* Lite** (G-2714), by Koenig and Likhachev (2002): it "implements the same navigation strategy as D* but is algorithmically different", built on A*-like keys and the lookahead rhs, and it is shorter and simpler to analyse (Koenig and Likhachev 2002). Current systems are typically based on D* Lite rather than the original D* (Wikipedia, D*).

On this small grid, the repair took 12 cells and A* from scratch 11. The repair is worth using only where it costs less than a new search.

4. When repairing costs less than a new search

> **Key point:** On the 0.5 m grid repairing costs as much as searching again. On finer maps of the same room, where the box is seen piece by piece, D* Lite's repairs cost 387 cells against 596 for A* from scratch, and a piece of the box behind the robot costs it nothing at all.

The opening problem asked for replanning that does not start over at every surprise. One box on a 76-cell grid is too small a test.

A real robot's map has finer cells, and its sensor sees an obstacle piece by piece, so it replans many times. We cut the same room into cells of 0.25 m and 0.1 m, let the robot see 0.5 m round itself, and count, at every replan, the cells D* Lite, the repairing planner of Section 3, takes off its queue and the cells A* from scratch takes off its open list from the robot's cell.

Here D* Lite starts, like A*, with no numbers: its first plan computes only the numbers it needs to reach S (notebook).

Cell side	Free cells	Replans	D* Lite: first plan, replans	A* from scratch: first plan, replans
0.5 m	76	1	12, then 19	9, then 11
0.25 m	304	2	32, then 67	26, then 73

Cell side	Free cells	Replans	D* Lite: first plan, replans	A* from scratch: first plan, replans
0.1 m	1900	7	134, then 387	117, then 596

On the 0.5 m grid the single repair costs D* Lite more than in Figure 2, 19 cells, because its first plan computed only the numbers on the way from S, and the way round the box needs numbers it never computed. As the cells shrink, the saving grows: 387 against 596 replanning cells on 0.1 m cells. Koenig and Likhachev (2002) find the same trend on random maps, D* Lite's "advantage seems to increase as the terrain gets larger", and report earlier results in which D* beat repeated A* "by one order of magnitude or more".

Figure 3 plays the 0.1 m drive. Watch the light red cells at each replan: in frame "move 3", D* Lite's first big repair (238 cells) costs more than A*'s new search (125), because every number it raises must later fall again; in frame "move 4", the next piece of the box costs D* Lite 44 cells and A* 125 again.

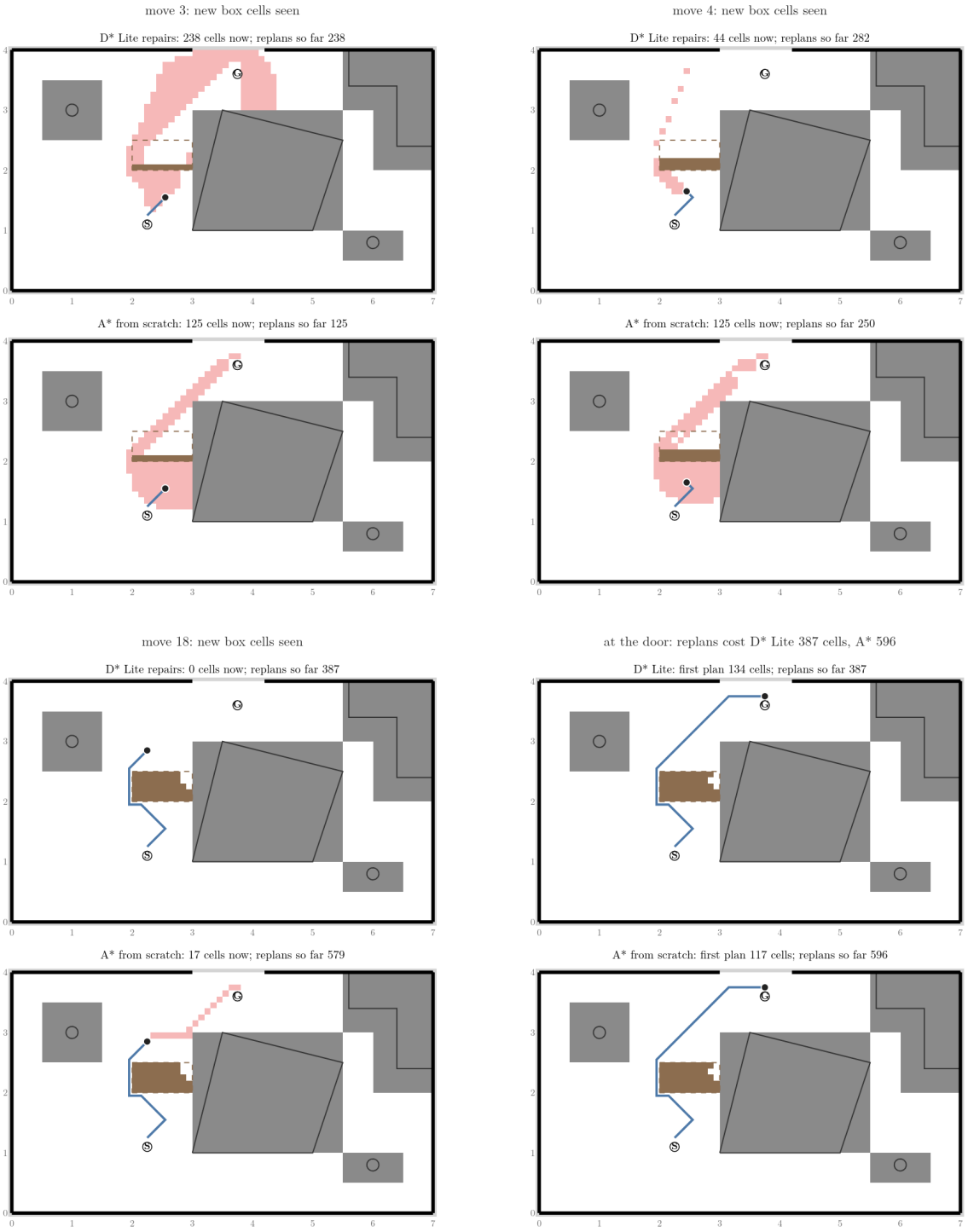


Figure 3: The same drive on the room cut into 0.1 m cells, one frame per move; the robot (black dot) sees 0.5 m round itself. Top: D* Lite, which keeps its numbers and repairs them; bottom: A* from scratch, run from the robot's cell at the same moments. Light red: the cells each method takes off its queue at this replan; brown: box cells seen so far; dashed: the box. The first plan costs D* Lite 134 cells and A* 117. At the 7 replans D* Lite takes 238, 44, 39, 35, 31, 0 and 0 cells, A* 125, 125, 105, 117, 90, 17 and 17: 387 against 596. The last two pieces of the box lie behind the robot, so no number it needs changes and D* Lite does no work

The last two replans cost D* Lite nothing: the newly seen box cells lie behind the robot, so no number on its way to the door depended on them. A* from scratch cannot know that and searches 17 cells each time. A repair touches only what a change affects; a fresh search computes the whole route every time.

A limitation. Searching from the door helps because only the robot moves. If the goal moves as well, numbers measured to the old goal are all wrong, and the CMU slides advise replanning from scratch in that case (CMU incremental, slide 9).

D* Lite still finishes a search before every move, up to the robot's own cell. On a large map that can take longer than the robot can wait.

5. No time to plan to the door: real-time search

> **Key point:** Real-time search limits each search to a fixed, small number of cells, moves, and searches again, raising its guess of the distance wherever the guess proved too low. LRTA* looks one cell ahead and escapes the pocket by the box after 4 raises, 9 moves, 5.121 m; RTAA* looks 4 cells ahead, 13 cells in 4 searches, 8 moves, 4.414 m.

5.1 Learning the guesses: LRTA*

The problem. Every method so far plans all the way to the door before the robot moves. On large terrains a search can take minutes, which "adds up to substantial idle times" (Koenig and Likhachev 2002). A robot that must keep moving needs a strict limit on how much it searches per move (CMU real-time, slide 3).

The idea. Look only at the cells around the robot, step to the one that looks best, and remember what was learned. The robot's guess of each cell's distance to the door starts as the heuristic (G-2570) of A*, the octile distance. Real-time search runs from the robot towards the door, so here the guess is of the distance to the door; the h in D* Lite's key of Section 3.2 guessed the distance to the robot, because D* Lite searches from the door. When every neighbour's offer is larger than the guess at the robot's cell, the guess was too low, and the robot raises it, so it does not return to this cell with the same too-low guess.

The standard term. Searching only a bounded number of cells around the robot before each move is **real-time heuristic search** (G-2715) (CMU real-time, slide 4). With a lookahead of one cell it is **LRTA***, learning real-time A* (G-2716) (Korf 1990; CMU real-time, slide 17). At each move (CMU real-time, slide 12):

1. set the guess at the robot's cell to its best offer, the smallest move plus neighbour's guess;
2. move to the neighbour that gives that smallest offer.

With:

- s is the robot's cell, and n one of its neighbours;
- $h(s)$ is the robot's current guess of the distance from s to the door, in m, at first the octile distance: 3.121 m for S;

- $c(s, n)$ is the length of the move from s to n, 0.5 m straight or 0.707 m diagonal, as in Section 3.1.

$$h(s) \leftarrow \min_n \Big(c(s, n) + h(n) \Big)$$

The arrow means: replace the guess on the left by the right side.

Step 1 is the value iteration (G-2704) update, done at one cell.

On our numbers. From S, two moves tie at 3.121: straight up, to a cell whose guess is 2.621, and diagonally up and right, to a cell whose guess is 2.414. Ties go to the smaller guess, as in A*, so the robot goes diagonally, into the pocket between the box and the table. There it sees the box, and its offers are (notebook):

Move from the pocket	Move (m)	Neighbour's guess (m)	Offer (m)
left	0.5	2.621	3.121
down	0.5	2.914	3.414
diagonally back to S	0.707	3.121	3.828

Every offer is larger than the cell's guess of 2.414: the cell is a dent in the guesses, a local minimum (G-1110), where a robot that only ever stepped to the lowest guess would stay forever (CMU real-time, slide 11). LRTA* raises the guess to 3.121 and moves left.

Figure 4 (left) plays the whole walk. Watch the orange outlines collect: 4 guesses rise (2.414 to 3.121, 2.621 to 3.328, 2.828 to 3.121, 2.621 to 2.914) until the guesses lead out round the box.

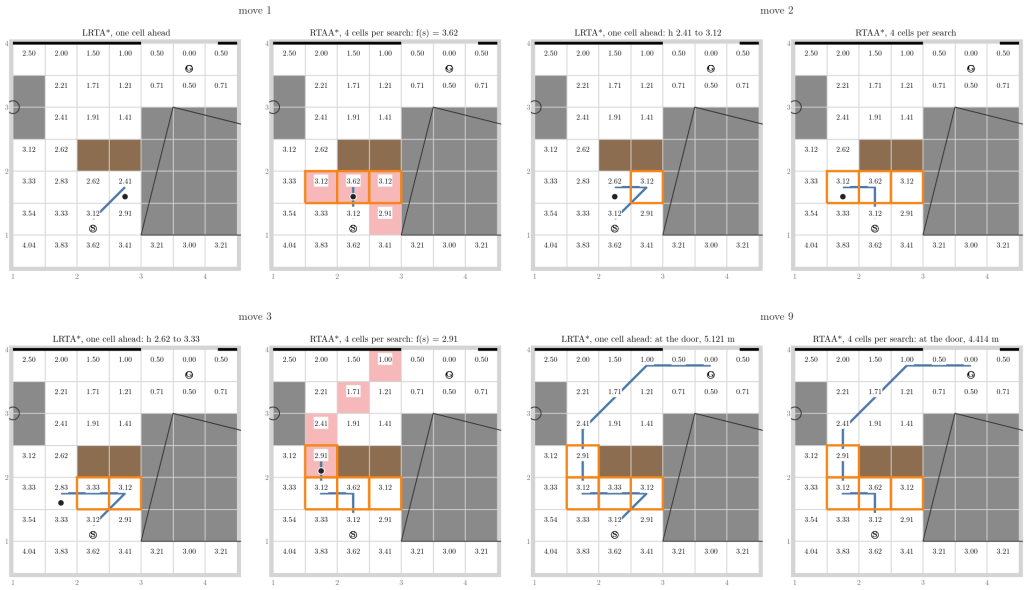


Figure 4: Real-time search with the box unknown until seen, on the part of the room the robot crosses (x 1.0 to 4.5 m, y 0.5 to 4.0 m), one move per frame. Each cell shows the robot's guess h of its distance to the door, starting as the octile distance; brown: box cells seen; orange outline: a guess that has been raised. Left: LRTA*, looking one cell ahead: it walks into the pocket between the box and the table, raises 4 guesses, and reaches the door after 9 moves, 5.121 m. Right: RTAA*, taking up to 4 cells off an A* open list per search (light red), setting each such cell's guess to f(s) − g, where s is the best cell left on the open list, f(s) its estimated total and g the cell's cost so far from the robot; it reaches the door after 4 searches of 13 cells in all, 8 moves, 4.414 m

LRTA* searches 9 cells in all, one per move, and drives 5.121 m (notebook).

Why raising a guess is safe. The guesses start as an admissible heuristic (G-2578), never above the true distance. The true distance from the pocket cell is its best move plus the true distance from there, and each true distance is at least its guess. With:

- s is the pocket cell, the robot's cell;
- $h^*(s)$ is its true distance to the door, 4.414 m, with the box known;
- n^* is the first cell of its true shortest route, the cell to its left, whose true distance is 3.914 m and whose guess is 2.621 m.

Our numbers first, then the same in symbols:

$$h^*(s) = 0.5 + 3.914$$

$$h^*(s) = c(s, n^*) + h^*(n^*)$$

$$0.5 + 3.914 \geq 0.5 + 2.621$$

$$c(s, n^*) + h^*(n^*) \geq c(s, n^*) + h(n^*)$$

$$0.5 + 2.621 \geq 3.121$$

$$c(s, n^*) + h(n^*) \geq \min_n \Big(c(s, n) + h(n)\Big)$$

So the raised guess, 3.121, is still at most the true 4.414: the guesses stay admissible, and they also stay consistent (G-2580) (CMU real-time, slide 14). LRTA* reaches the goal in a finite number of moves when every move costs more than 0, the map is finite with a route to the goal, and every move can be undone (CMU real-time, slide 15).

5.2 Looking further: RTAA*

LRTA* sees one cell ahead and learns one guess per move, so it found the pocket only by walking into it. Looking further, through more cells per move, lets each search learn many guesses at once (CMU real-time, slides 34 to 36).

The idea. Run A* from the robot's cell, but stop after N cells have come off the open list. The best cell left on the open list, s, is where the robot heads. Every cell u that came off learns from s in one subtraction. With:

- $g(u)$ is A*'s cost so far from the robot's cell to u, in m;
- $f(s)$ is the smallest estimated total on the open list, the best cell's cost so far plus its guess, in m.

$$h(u) \leftarrow f(s) - g(u)$$

The robot then moves along the found path towards s. The method is **RTAA***, real-time adaptive A* (G-2717) (Koenig and Likhachev 2006; CMU real-time, slide 34). LRTA* with a lookahead of N cells would update the same cells by value iteration, which is more informed but slower; RTAA*'s update is one pass (CMU real-time, slide 39).

On our numbers, with N = 4. The second search starts at the cell left of the robot's pocket, x 2.0 to 2.5 m, y 1.5 to 2.0 m, after the box has been seen. The search takes off that cell, the pocket, the cell to its left and the cell below the pocket; the best cell left is the cell left of the box, 1.0 m away with a guess of 2.621 (notebook):

$$f(s) = 1.0 + 2.621 = 3.621$$

Each cell that came off gets the new guess: the best total minus its own cost so far.

Cell taken off	g (m)	New guess $f(s) - g$ (m)	Old guess (m)
its own cell	0	3.621	2.621
the pocket	0.5	3.121	2.414
the cell to its left	0.5	3.121	2.828
the cell below the pocket	0.707	2.914	2.914, unchanged

Why the update is safe (CMU real-time, slide 38). Any route from the robot to the door that passes through u costs at least the shortest, and A* with an admissible heuristic has $f(s)$ at most the shortest route's length (the A* bound). With:

- r is the robot's cell, whose true distance $h^*(r)$ to the door is 3.914 m;
- u is the pocket, with cost so far $g(u)$ of 0.5 m and true distance $h^*(u)$ of 4.414 m.

Our numbers first, then the same in symbols:

$$0.5 + 4.414 \geq 3.914 \geq 3.621$$

$$g(u) + h^*(u) \geq h^*(r) \geq f(s)$$

$$4.414 \geq 3.621 - 0.5 = 3.121$$

$$h^*(u) \geq f(s) - g(u)$$

So the new guess never exceeds the true distance. The whole drive takes 4 searches of at most 4 cells, 13 cells in all, and 8 moves, 4.414 m (notebook; Figure 4, right). Its first move, straight up from S, is not foresight: the box is still unseen, and every cell of the first search ties with the cell straight up at an estimated total of 3.121 m, so that cell happens to be the best one left on the open list. On a map this small such ties have a large effect: with $N = 2$ the drive is 5.828 m, and with $N = 8$ it is 5.121 m (notebook). Over many maps, a smaller lookahead gives longer drives but fewer cells per search (Koenig and Likhachev 2006).

Both methods still need the robot's grid map, with a guess stored for every cell they have visited.

6. No map at all: bug algorithms

> **Key point:** A robot that knows only where the door is and what it touches can still get there: head straight for the door, and follow the edge of whatever blocks the way. Bug 1 goes round the whole obstacle before leaving at the point nearest the door, 16.912 m; Bug 2 leaves as soon as it is back on the straight line to the door and nearer than where it hit, 4.885 m.

The problem. Every method so far needs a grid map: numbers, or at least guesses, for every cell. We place the robot in a room it has never seen, with the box where it was.

The robot knows two things: the direction and distance of the door, from a goal sensor, and the shape of an obstacle it touches, from a short-range sensor (LaValle §12.3.3). Everything else, the table too, is unknown.

The robot as a point. We follow the robot's centre and grow every obstacle by the robot's radius, 0.25 m, the grown obstacles (G-2652) of configuration space: then the centre may touch a grown obstacle's edge but never cross it. The gap between the box and the table, 0.25 to 0.375 m, is narrower than the robot's 0.5 m width, so the two grown shapes join into one obstacle, 11.702 m round (notebook).

The standard terms. A strategy that heads for the goal and follows the edge of any obstacle it meets, using only a goal sensor and contact, is a **bug algorithm** (G-2718) (LaValle §12.3.3; Platt bugs). Our robot starts at (2, 1) and heads for the middle of the doorway, (3.6, 4.0), 3.4 m away; both algorithms below turn left when they hit, so the obstacle stays on the robot's right.

Bug 1 (Lumelsky and Stepanov 1987; LaValle §12.3.3):

1. move straight towards the goal until it reaches the goal or hits an obstacle;
2. follow the obstacle's whole edge back to the hit point, remembering the point nearest the goal;
3. return to that point along the edge, the shorter way, and go to step 1.

Bug 2 (LaValle §12.3.3; Platt bugs):

1. move along the start–goal line, the straight line from the start to the goal, until it reaches the goal or hits an obstacle;
2. follow the obstacle's edge until it meets the start–goal line again at a point nearer the goal than the hit point;
3. leave the edge there and go to step 1.

Figure 5 runs both. Watch the hit point H, where both meet the obstacle after 0.850 m, then Bug 1's full loop past C, the point nearest the door, against Bug 2's early exit at L.

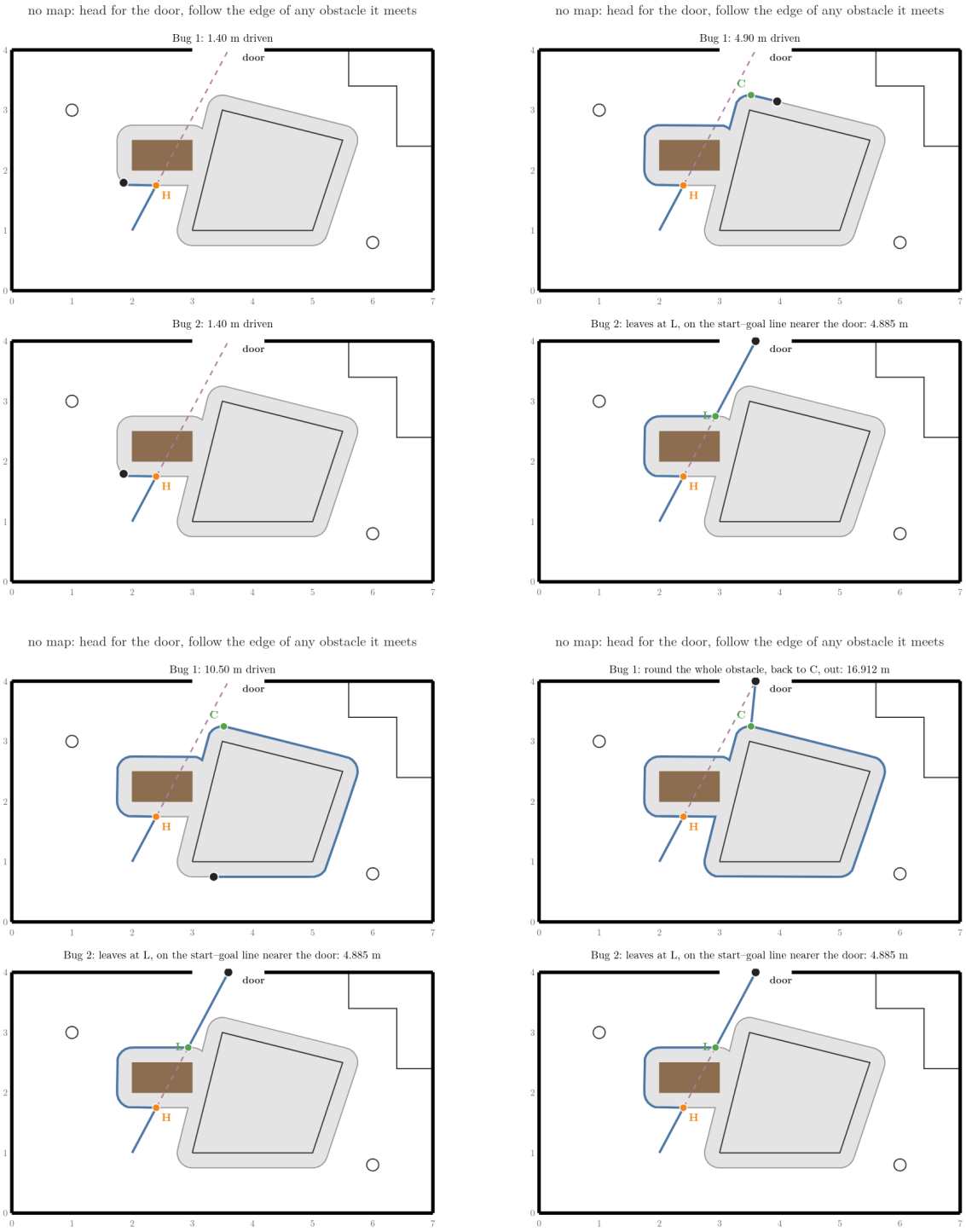


Figure 5: Bug 1 (top) and Bug 2 (bottom) in the room with no map: the robot's centre (black dot) starts at (2, 1) and heads for the middle of the doorway, (3.6, 4.0). The box (brown) and the table (outline) are grown by the robot's radius, 0.25 m (pale grey band); the gap between them is narrower than the robot, so the two grown shapes form one obstacle, 11.702 m round. Dashed: the start–goal line, from start to door. Both hit the obstacle at H, 0.850 m from the start, and turn left. Bug 1 goes once round the whole obstacle, passing C, the point nearest the door (0.755 m from it), returns to C the shorter way, 3.604 m, and leaves: 16.912 m. Bug 2 leaves at L, where it meets the start–goal line again 1.417 m from the door (H is 2.550 m from it): 4.885 m

Bug 1 on our numbers. The robot drives 0.850 m to H, 11.702 m round the obstacle, 3.604 m back to C, and 0.755 m from C to the door. To 4 decimals (notebook):

$$0.8500 + 11.7023 + 3.6043 + 0.7550$$

$$= 16.912 \text{ m}$$

Bug 1 knows that it has gone all the way round when the goal sensor's reading repeats, and it can find the shorter way back because the readings tell it where it is (LaValle §12.3.3).

Bug 2 on our numbers. H is 2.550 m from the door. Bug 2 follows the edge for 2.619 m, to L on the start–goal line, 1.417 m from the door, and leaves:

$$0.8500 + 2.6187 + 1.4167$$

$$= 4.885 \text{ m}$$

The worst case. With:

- d is the straight-line distance from start to goal, 3.4 m;
- p_i is the length round obstacle i, here one obstacle with $p = 11.702$ m;
- n_i is the number of times obstacle i's edge crosses the start–goal line, here 2, at H and L;
- $\sum_i$ adds up what follows over every obstacle i, here one obstacle only.

LaValle (§12.3.3) gives the longest each can drive. Bug 1 follows each edge at most one and a half times:

$$\text{Bug 1} \leq d + \tfrac{3}{2} \sum_i p_i$$

$$= 3.4 + 17.553 = 20.953 \text{ m}$$

Bug 2 follows each edge at most half its length per crossing:

$$\text{Bug 2} \leq d + \tfrac{1}{2} \sum_i n_i \, p_i$$

$$= 3.4 + 11.702 = 15.102 \text{ m}$$

Our runs, 16.912 m and 4.885 m, stay inside both limits. Bug 2 is far shorter here, but not always: an obstacle whose edge crosses the start–goal line many times makes Bug 2 follow it piece after piece, each crossing adding to its bound (LaValle §12.3.3, Figure 12.23).

Bug 1 drives its long loop to see every point of the edge before choosing where to leave. Both assume the robot always knows its position exactly and moves exactly as commanded (Platt bugs).

7. Summary

Method	Needs	On the box drive	Why use it
A* from scratch at each surprise	a map; a full search per replan	11 cells (0.5 m); 596 over 7 replans (0.1 m)	simple, but every search starts from nothing
D* / D* Lite	a map; numbers kept from a backward search	12 repair steps; 387 over 7 replans (0.1 m)	repairs only the numbers a change affects
LRTA*	a map of guesses; 1 cell searched per move	9 moves, 5.121 m	a fixed, small amount of search per move
RTAA*	a map of guesses; N cells per move	4 searches, 13 cells, 4.414 m	looks further; one subtraction per cell to learn
Bug 1	goal direction and contact only	16.912 m	sees the whole edge first; at most $d + \frac{3}{2}\sum p_i$
Bug 2	goal direction, contact, the start–goal line	4.885 m	leaves early; at most $d + \frac{1}{2}\sum n_i p_i$

Here d is the start's straight-line distance to the goal, p_i the length round obstacle i and n_i the number of times its edge crosses the start–goal line.

- The unknown box made the robot's cell's number wrong, 2.707 m where the way round is 4.414 m, so the robot must replan; it treats every cell it has not seen blocked as free, so its plans are never too pessimistic, and **replanning** (G-2711) fixes them when they are too optimistic.
- Planning from the door keeps the numbers right while the robot moves, because they measure the distance to the door, which does not move; only the cells whose route ran through the box go wrong.
- **D*** (G-2713) fixes only the cells whose number differs from their lookahead, the smallest move plus neighbour's number, in key order, raising the ones that relied on the change and lowering them to the new way round, so 12 steps reach the robot's true 4.414 m and 68 of 76 numbers are never touched.
- **D* Lite** (G-2714) gives D*'s behaviour with A*-like keys and the lookahead rhs; it saves work when the map is large and the changes are seen piece by piece: 387 against 596 replanning cells on 0.1 m cells, and nothing at all for box cells behind the robot.
- **Real-time heuristic search** (G-2715) bounds the search per move; **LRTA*** (G-2716) raises the guess at a cell whose every offer is worse, so it escapes the pocket by the box after 4 raises, and the guesses stay admissible because a true distance is at least any guessed offer.
- **RTAA*** (G-2717) looks a fixed number of cells ahead and sets each looked-at cell's

guess to the best estimated total left on the open list minus the cell's own cost so far, which stays admissible because that total never exceeds the shortest route; looking 4 cells ahead, it reaches the door in 4.414 m after 13 cells.

- A **bug algorithm** (G-2718) needs no map: Bug 1 circles the whole obstacle before leaving, 16.912 m, so it never misses the best exit; Bug 2 leaves where the start–goal line crosses the edge nearer the door, 4.885 m, but can be longer on an obstacle that crosses the start–goal line many times.

The robot can now reach the door when its map is wrong, when it has little time to plan, and when it has no map at all. The methods here still move on a grid or follow obstacle edges; a potential field builds a downhill surface straight from the obstacles' shapes, with no grid at all.

8. Sources

Built from

- CSMinute, "Intro to Path Planning: D* Lite vs. A*", YouTube, https://www.youtube.com/watch?v=sk 3UfcXW0
- MIT OpenCourseWare, "Advanced 1. Incremental Path Planning" (MIT 16.412J Cognitive Robotics), YouTube, https://www.youtube.com/watch?v=_4u9W1xOuts
- Likhachev, M. (Carnegie Mellon University), *16-350 Planning Techniques for Robotics*, slides "Interleaving Planning and Execution: Incremental Heuristic Search", https://www.cs.cmu.edu/~maxim/classes/robotplanning/lectures/incrementalsearch_16350_sp26.pdf (CMU incremental), and "Interleaving Planning and Execution: Real-time Heuristic Search", https://www.cs.cmu.edu/~maxim/classes/robotplanning/lectures/realtimesearch_16350_sp2 (CMU real-time)
- Platt, R. (Northeastern University), *CS5335 Robotic Science and Systems*, slides "Path planning: BUGs and wavefront", https://www.ccs.neu.edu/home/rplatt/cs5335_2016/slides/bugs_wa (Platt bugs)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §12.3.2 "Stentz's algorithm (D*)" and §12.3.3 "Planning in unknown continuous environments" (the Bug1 and Bug2 strategies). Free online: https://lavalle.pl/planning/node616.html, https://lavalle.pl/planning/node618.html, https://lavalle.pl/planning/node619.html and https://lavalle.pl/planning/node620.html (LaValle)
- Koenig, S. and Likhachev, M. (2002). "D* Lite". *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-02)*, http://idm-lab.org/bib/abstracts/papers/aaai02b.pdf (Koenig and Likhachev 2002)

Other references

- Stentz, A. (1994). "Optimal and efficient path planning for partially-known environments". *Proceedings of the IEEE International Conference on Robotics and Automation*, 3310–3317 (Stentz 1994)
- Korf, R. E. (1990). "Real-time heuristic search". *Artificial Intelligence* 42(2–3), 189–211 (Korf 1990)

- Koenig, S. and Likhachev, M. (2006). "Real-time adaptive A*". *Proceedings of the Fifth International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 281–288 (Koenig and Likhachev 2006)
- Lumelsky, V. J. and Stepanov, A. A. (1987). "Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape". *Algorithmica* 2, 403–430 (Lumelsky and Stepanov 1987)
- Wikipedia, "D*", https://en.wikipedia.org/wiki/D%2A (Wikipedia, D*)

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Replanning (G-2711)	Computing a new plan while the robot is moving, because its map, its position or the world has changed, such as a box seen on the route; it keeps the robot on a route that is still possible.
D* (dynamic A*) (G-2713)	A replanner that keeps the cost-to-go from a backward search and, when sensed costs differ from the map, puts the affected cells back in a priority queue and fixes them until the robot's own cost-to-go is right again; it repairs only the numbers a change affects instead of searching from scratch.
D* Lite (G-2714)	A simpler algorithm with the same behaviour as D*, built on A*-like keys and a one-step lookahead (rhs) for every cell; current D* systems are typically based on it.
Real-time heuristic search (G-2715)	Searching only a fixed, small number of states around the robot before each move, then moving and searching again; it bounds the search time per move at the cost of a possibly longer drive.
LRTA* (learning real-time A*) (G-2716)	Real-time search that looks one move ahead: it raises the heuristic at the robot's state to its best offer, the smallest move plus neighbour's heuristic, and moves there; the raised guesses stay admissible and lead the robot out of dead ends.

Term	Meaning
RTAA* (real-time adaptive A*) (G-2717)	Real-time search that takes up to N states off an A* open list, sets each one's heuristic to f(s) minus its cost so far, s being the best state left on the open list, and moves towards s; one subtraction per state keeps the guesses admissible.
Bug algorithm (G-2718)	A strategy for a robot with no map that knows only the goal's direction and what it touches: head for the goal, and follow the edge of any obstacle met until a rule says to leave (Bug 1: at the nearest point after a full loop; Bug 2: back on the straight start–goal line, nearer the goal).
Cost-to-go (G-2701)	The length of the shortest route from a state to the goal, such as 3.414 m from the room's start cell to the door; stored for every state, it tells the robot how far it still has to go from wherever it is.
A* search (G-2577)	Dijkstra's algorithm with the open list sorted by $f = g + h$, the cost so far plus a heuristic guess of the cost left; with a consistent heuristic it returns the shortest route while expanding far fewer states (9 cells against 54 on the room grid).
Backward search (G-2583)	A search that starts at the goal and follows every move backwards until it reaches the start; it is cheaper when the goal's side branches less, and run to the end it gives every state's cost to the goal.
Value iteration (G-2704)	Computing every state's cost-to-go by repeating, over all states, the update "value = smallest move cost plus the next state's value" until no value changes; it needs no priority queue and no order, and it carries over to values between grid cells and to uncertain moves.
Priority queue (G-2568)	A collection with two operations, insert an entry with a priority and remove the entry with the best priority (for Dijkstra's algorithm, the smallest distance); it is what lets a search always pick its most promising vertex next.

Term	Meaning
Dijkstra's algorithm (G-2565)	A search that always settles the vertex with the smallest distance from the start found so far, which is then final, and offers each of its neighbours the route through it, keeping the shorter; with every edge cost at least 0 it finds the shortest route to every vertex (3.414 m to the door on the room grid with diagonals).
Octile distance (G-2571)	The length of the shortest route between two cells of an 8-connected grid with no obstacles: diagonal moves while both the column and the row count shrink, then straight moves; with straight moves of length a and diagonals of length ℓ it is $\ell \min + a(\max - \min)$ of the column and row counts; a heuristic for 8-connected grids that never overestimates.
Heuristic h (G-2570)	A cheap guess of the cost still to go from a state to the goal, such as 3.121 m from the start cell to the door; a search uses it to prefer states that look nearer the goal.
Local minimum (G-1110)	A point lower than everything around it, but not the lowest overall.
Admissible heuristic (G-2578)	A heuristic that is never more than the true cost left from any state, $h \leq h^*$; it makes f a lower bound on every route through a state, so the first route A* takes off the open list is the shortest.
Consistent heuristic (monotone heuristic) (G-2580)	A heuristic that never falls by more than a move's cost, $h(n) \leq c(n, n') + h(n')$, with 0 at the goal (a triangle inequality); it keeps f from decreasing along a route, so A* with a closed list expands every state with its shortest g.
Grown obstacle (inflated obstacle) (G-2652)	An obstacle with every point within the robot's radius added, its edges pushed out and its corners rounded; for a disk robot it is exactly the obstacle region, so the robot can be treated as a point (the table, 3.625 m^2, grows to 5.747 m^2 for a 0.25 m radius).